

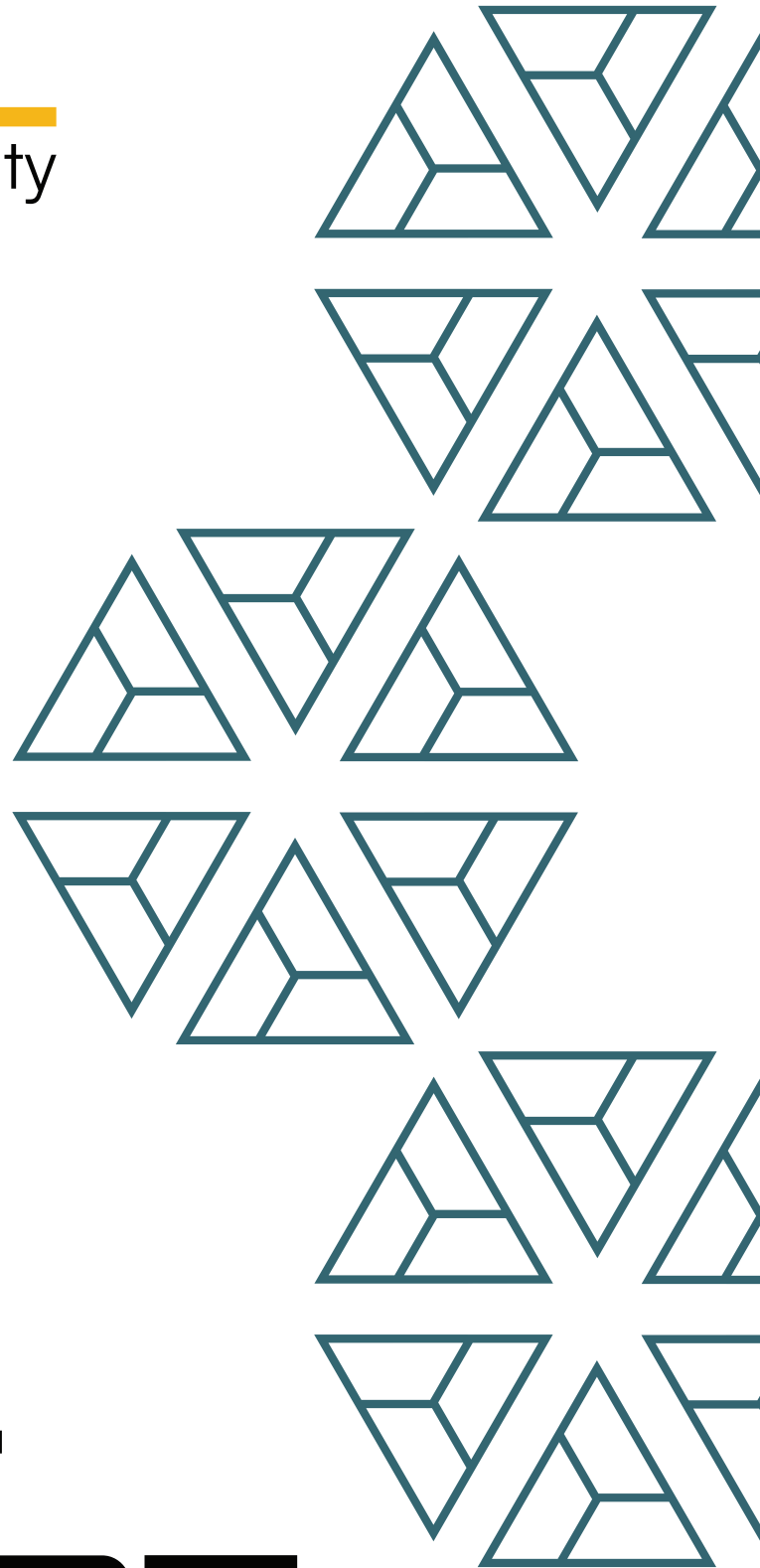


BAIL
security

ListaDao
SlisBNB

FINAL REPORT

August '2025



Disclaimer:

Security assessment projects are time-boxed and often reliant on information that may be provided by a client, its affiliates, or its partners. As a result, the findings documented in this report should not be considered a comprehensive list of security issues, flaws, or defects in the target system or codebase.

The content of this assessment is not an investment. The information provided in this report is for general informational purposes only and is not intended as investment, legal, financial, regulatory, or tax advice. The report is based on a limited review of the materials and documentation provided at the time of the audit, and the audit results may not be complete or identify all possible vulnerabilities or issues. The audit is provided on an "as-is," "where-is," and "as-available" basis, and the use of blockchain technology is subject to unknown risks and flaws.

The audit does not constitute an endorsement of any particular project or team, and we make no warranties, expressed or implied, regarding the accuracy, reliability, completeness, or availability of the report, its content, or any associated services or products. We disclaim all warranties, including the implied warranties of merchantability, fitness for a particular purpose, and non-infringement.

We assume no responsibility for any product or service advertised or offered by a third party through the report, any open-source or third-party software, code, libraries, materials, or information linked to, called by, referenced by, or accessible through the report, its content, and the related services and products. We will not be liable for any loss or damages incurred as a result of the use or reliance on the audit report or the smart contract.

The contract owner is responsible for making their own decisions based on the audit report and should seek additional professional advice if needed. The audit firm or individual assumes no liability for any loss or damages incurred as a result of the use or reliance on the audit report or the smart contract. The contract owner agrees to indemnify and hold harmless the audit firm or individual from any and all claims, damages, expenses, or liabilities arising from the use or reliance on the audit report or the smart contract.

By engaging in a smart contract audit, the contract owner acknowledges and agrees to the terms of this disclaimer.

1. Project Details

Important:

Please ensure that the deployed contract matches the source-code of the last commit hash.

Project	ListaDao - SlisBNB
Website	lista.org
Language	Solidity
Methods	Manual Analysis
Github repository	https://github.com/lista-dao/synclub-contracts/tree/a3524827a96be797151d154669c3cb023b6615d3
Resolution 1	https://github.com/lista-dao/synclub-contracts/tree/ad380197fe4ad204f5c5f048f100783043d5190f

2. Detection Overview

Severity	Found	Resolved	Partially Resolved	Acknowledged (no change made)	Failed resolution	Open
High						
Medium	7	2		5		
Low	12	3		9		
Informational	4	2		2		
Governance						
Total	23	7		16		

2.1 Detection Definitions

Severity	Description
High	The problem poses a significant threat to the confidentiality of a considerable number of users' sensitive data. It also has the potential to cause severe damage to the client's reputation or result in substantial financial losses for both the client and the affected users.
Medium	While medium level vulnerabilities may not be easy to exploit, they can still have a major impact on the execution of a smart contract. For instance, they may allow public access to critical functions, which could lead to serious consequences.
Low	Poses a very low-level risk to the project or users. Nevertheless the issue should be fixed immediately
Informational	Effects are small and do not post an immediate danger to the project or users
Governance	Governance privileges which can directly result in a loss of funds or other potential undesired behavior

3. Detection

SLisLibrary

The [SLisLibrary](#) provides functionality to calculate daily and annual fees from daily profits and an annual principal amount based on synFee and annualRate percentages. This functionality is utilized by the ListaStakeManager when BNB profits are realized.

Core Invariants:

INV 1: If `feeFromAPY` is greater than `feeFromProfit`, `calculateFee()` should always return `feeFromAPY`.

INV 2: If `feeFromProfit` is greater than or equal to `feeFromAPY`, `calculateFee()` should always return `feeFromProfit`.

Privileged Functions

- None

No issues found.

ListaStakeManager

The [ListaStakeManager](#) handles BNB liquid staking on the BSC chain through the native Stake Hub contract. The contract provides end users with the option to deposit BNB to the contract and withdraw it either instantly by paying a withdrawal fee or requesting a withdrawal that is unlocked after 7 days.

Interactions with the Stake Hub contract are restricted to an access-controlled BOT role that has the ability to delegate, redelegate and undelegate BNB as well as compound unrealized rewards in the contract.

Appendix: Undelegations

Undelegation is the process by which the BOT initiates the release of staked BNB from a validator. When the BOT specifies an operator and an amount, the contract first checks that this amount does not exceed what can be safely undelegated. The check explicitly allows `_amount` to be at most the sum of `getAmountToUndelegate()` (the amount currently needed to cover the withdrawal queue, minus what is already unbonding or available as undelegated quota) and `reserveAmount`. The inclusion of `reserveAmount` provides a safety buffer, because undelegation is share based, the actual BNB returned from [StakeHub](#) is often slightly smaller than the requested amount due to precision loss. Without this buffer, even tiny precision loss would cause user withdrawals to fail. By allowing an additional reserve defined by `reserveAmount`, the contract ensures that these discrepancies can be resolved correctly. Once validated, the requested amount is converted into shares, and the corresponding shares are undelegated via StakeHub. The contract records the actual BNB amount that will be released and increments `unbondingBnb` until the funds are later claimed. In this way the buffer logic guarantees withdrawals remain claimable even under minor rounding mismatches.

Appendix: Refund commissions

Refund commission is the mechanism by which Lista DAO can return validator commission fee to stakers in a linear, time-distributed manner. The [MANAGER](#) initiates this process by sending BNB to the contract along with a duration, expressed in days. The contract converts the BNB into `slisBNB`, mints it to itself, and immediately increases `amountToDelegate` so that the refund is restaked into the [StakeHub](#). Instead of distributing the refund in one iteration, the contract schedules a daily burn of `slisBNB`, which burns the daily calculated `slisBnb` amount (if triggered 24 hours after the last burn) which increases the `slisBnb` value over time. The schedule is tracked using the [Refund struct](#), which records how much `slisBNB` should be

burned each day [`dailySlisBnb`], how much remains to be burned [`remainingSlisBnb`], and the last time a burn occurred [`lastBurnTime`] which is only updated if the burnAmount was greater than 0 i.e. `remainingSlisBnb > 0`.

Core Invariants:

INV 1: The sum `totalDelegated + amountToDelegate` should always equal `getPooledBnb()`.

INV 2: Each `WithdrawalRequest.uuid` should always be greater than the previous one(`requestUUID` is monotonically increasing).

INV 3: If `uuid < withdrawalQueue[0].uuid`, then the request should always be treated as an old withdrawal request i.e. handled by the old logic branch.

INV 4: `nextConfirmedRequestUUID` should always equal to the uuid of the first unconfirmed request(or be strictly greater than the last request's uuid if all are confirmed).

INV 5: If `validators[addr] == true`, then that validator should always have been added and synced with `syncCredits`.

INV 6: `refund.lastBurnTime` should only ever update when `burnAmount > 0`.

INV 7: The sum `undelegatedQuota + unbindingBnb + totalDelegated` should always be greater than or equal to the sum of all `withdrawalQueue[i].amount`.

Privileged Functions

- `delegateTo()`
- `redelegate()`
- `claimWithdrawFor()`
- `undelegateFrom()`
- `claimUndelegated()`
- `delegateVoteTo()`
- `depositReserve()`
- `withdrawReserve()`
- `claimWithdrawFee()`
- `setReserveAmount()`
- `setSynFee()`
- `setMinBnb()`

- `setRedirectAddress()`
- `setRevenuePool()`
- `whitelistValidator()`
- `disableValidator()`
- `removeValidator()`
- `setMaxBufferSizePct()`
- `setInstantWithdrawFeeRate()`
- `pause()`
- `unpause()`
- `toggleVote()`
- `compoundRewards()`
- `refundCommission()`

Issue_01	<code>compoundRewards</code> can be sandwiched for risk free profits
Severity	Medium
Description	When <code>compoundRewards()</code> is called by the BOT the function calculates the total profit made by the validators and increments the <code>totalDelegated</code> value , this in turn increases the <code>slisBnb</code> share value for the share holders. An attacker can therefore sandwich a <code>compoundRewards()</code> call by depositing BNB just before the <code>compoundRewards()</code> call then <code>compoundRewards</code> would increment the share value of the attacker [also the <code>_burnRefundSlisBnb()</code> would increment share value] and then the attacker can backrun the <code>compoundRewards</code> call and call <code>requestWithdraw()</code> and since the <code>bnbToWithdraw</code> is fixed at <code>requestWithdraw</code> till the user claims the attacker has made a risk free profit.
Recommendations	Consider having a minimum period during which users can't request a withdrawal after a deposit.
Comments / Resolution	Acknowledged.

Issue_02	Unaccounted redelegate fee rate leads to inflated totalDelegated amount
Severity	Medium
Description	Function redelegate() can be used by the BOT role to change BNB delegations from one validator to another. During this process, the BNB Stake Hub charges a fee percent. For example, if 1.1 BNB is attempted to be redelegated from validator A to B, only 1 BNB is actually delegated to B. Currently the redelegate fee rate is 2/100000 (0.002%) on Stake Hub and is configurable. Since in the ListaStakeManager, the totalDelegated amount is the same (1.1 BNB from the above example), it becomes overinflated than the actual amount delegated to validators. To note, once profits are compounded, the totalDelegated will start reflecting its true value again. However, during the period between the redelegation (t1) and the next time rewards are compounded (t2), the totalDelegated will remain overinflated. This affects specific functionality in the ListaStakeManager that relies on correct accounting of totalDelegated. For example, totalDelegated is used in the function totalPooledBnb(), which is used by the convertBnbToSnBnb() and convertSnBnbToBnb() functions. Since totalDelegated is inflated, the exchange rate drifts as well. This means for every SnBnb, the amount of BNB is more than intended. This could be problematic in the following scenarios when withdrawals are requested during this period (t1 to t2) First Scenario: In instantWithdraw(), since convertSnBnbToBnb() will return an inflated BNB amount, this could be used to take more BNB from the protocol. Second Scenario:

	In requestWithdraw(), the BNB amount stored in the request will be higher than intended since convertBnbToSnBnb() returns an inflated amount. We assume this is the last request. The BOT now calls undelegateFrom(), however it will need to pass the BNB amount to be undelegated to be lesser since the request amount is inflated and the validator in reality holds lesser BNB. In claimUndelegated(), all remaining BNB from the validator will be claimed and stored in undelegatedQuota. However due the check on Line 422 (as seen below), the check would compare the inflated request totalAmount to the undelegatedQuota. This would brick the withdrawal for the last request. <pre><i>if (totalAmount <= undelegatedQuota) {</i></pre> This issue would also occur in a normal scenario where there are multiple users having their BNB delegated to that specific validator. However, that would only lead to a temporary DOS as reward compounding will adjust the totalDelegated back to the correct value and another withdrawal request would be needed to call claimUndelegated() and execute withdrawals.
Recommendations	Consider implementing a fix such that the exchange rate remains stable in such a scenario.
Comments / Resolution	Acknowledged.

Issue_03	<code>delegateVoteTo</code> can be DoSed if an attacker delegates vote to <code>ListaStakeManager</code>
Severity	Medium
Description	When delegating voting power there are two options , either the <code>delegateTo</code> address is the current contract address or any other address. When the <code>delegateTo</code> address is not <code>address(this)</code> → <pre> if (_delegateTo != address(this)) { require(govToken.getVotes(address(this)) == 0, "Invalid Delegation"); uint256 currDelegateeChange = currentVotePower - govToken.getVotes(currentDelegatee); uint256 newDelegateeChange = govToken.getVotes(_delegateTo) - newVotePower; require(currDelegateeChange == newDelegateeChange && balance == currDelegateeChange, "Invalid Change"); } </pre> The condition <code>require(govToken.getVotes(address(this)) == 0)</code> can be manipulated where an attacker can delegate voting power to the <code>ListaStakeManager</code> contract and due to this <code>govToken.getVotes(address(this)) > 0</code> and the function execution would revert. The same could be done for self delegation where the check is → <pre> else { require(govToken.getVotes(address(this)) == balance, "Self- delegation Failed"); } </pre> <code>getVotes</code> can be made to return a higher value than the <code>govBNB</code> balance of the <code>ListaStakeManager</code> contract.
Recommendations	Require statements can be removed or relaxed.
Comments / Resolution	Fixed by removing require statements.

Issue_04	Withdrawals can be bricked due to transfer gas limit on stake credit claims
Severity	Medium
Description	Currently on claiming unbonded requests, the BNB from the validator credit contract is received using the receive() function in the ListaStakeManager contract. The receive function however has code in it as seen in the snippet below. <pre> receive() external payable { if ([!creditStates[msg.sender]] && msg.sender != redirectAddress) { AddressUpgradeable.sendValue(payable(redirectAddress), msg.value); } } </pre> The issue with this is that the claim() function in the StakeCredit contract transfers the BNB to the stake manager using a low-level call and most importantly with a fixed gas limit as seen in the snippet below. <pre> uint256 _gasLimit = IStakeHub[STAKE_HUB_ADDR].transferGasLimit(); (bool success,) = delegator.call{ gas: _gasLimit, value: _totalBnbAmount }(""); if (!success) revert TransferFailed(); </pre> Currently, this gas limit is 5000, however it can be updated by the governance in StakeHub to any value between 2300 and 10000. <pre> uint256 newTransferGasLimit = value.bytesToUint256[32]; if (newTransferGasLimit < 2300 newTransferGasLimit > 10_000) revert InvalidValue(key, value); transferGasLimit = newTransferGasLimit; </pre>

First scenario: After running the below POC in `ListaStakeManager.t.sol` using `forge test -mt test_receive -vvvvv`, the call just makes it through (with 4557 gas as seen in the traces) in the case where the crediting contract is whitelisted (i.e. we do not enter the if block of the receive function). However, this cannot be relied upon since the gas limit can be configured in the Stake Hub and decreased anytime, in which case withdrawals will be bricked.

```
function test_receive() public {
    vm.deal(admin, 10 ether);

    // setting admin as the credit contract for simplicity
    vm.mockCall(
        STAKE_HUB,
        abi.encodeWithSignature("getValidatorCreditContract(address)",
        validator_A), abi.encode(admin)
    );

    vm.prank(admin);
    stakeManager.whitelistValidator(validator_A);

    vm.prank(admin);

    [bool success,] = address(stakeManager).call{value: 10 ether,
    gas: 5000}("");
}
```

Second scenario: The call will revert in the case the crediting contract is non-whitelisted and we enter the if block of the receive function which would attempt to make an external call to the redirect address that would consume more gas than provided by the crediting contract. This can be checked with the following POC below, where the receive function would consume 9999 gas (as seen in the traces) that is greater than the limit of 5000 gas. This scenario can occur in the case a redelegation occurred to a `dstValidator` that is not whitelisted and we attempt to withdraw from its non-whitelisted credit contract. The possibility of redelegations

	to a non-whitelisted dstValidator has been demonstrated in another issue in this report titled - "Function redelegate() does not validate dstValidator whitelist". <pre> function test_receive() public { vm.deal(user_A, 10 ether); // setting admin as the credit contract for simplicity vm.mockCall(STAKE_HUB, abi.encodeWithSignature("getValidatorCreditContract(address)", validator_A), abi.encode(admin)); vm.prank(admin); stakeManager.whitelistValidator(validator_A); vm.prank(user_A); [bool success,] = address(stakeManager).call{value: 10 ether, gas: 5000}(""); } </pre>
Recommendations	We recommend decreasing the amount of gas consumed by the receive function. One potential gas reduction could be to remove the redirect address condition and the external call from it. For extreme precaution, we recommend performing extensive testing on the gas consumed by the receive function to ensure it never consumes more than 2300 gas.
Comments / Resolution	Fixed by removing all logic from the receive() function.

Issue_05	fee shares is calculated incorrectly inside <code>compoundRewards</code>
Severity	Medium
Description	fee shares is calculated using <code>convertBnbToSnBnb</code> after adding the entire profit [this includes fees] to the <code>totalDelegated</code> variable <pre> uint256 totalProfit = totalBNBInValidators + undelegatedQuota - totalDelegated; uint256 fee = SLisLibrary.calculateFeeFromDailyProfit(totalProfit, synFee, TEN_DECIMALS); totalDelegated += totalProfit; uint256 slisBNBAmount = convertBnbToSnBnb(fee); </pre> This is flawed as adding the fees to <code>totalDelegated</code> incorrectly increases the share price causing the calculated feeShare to redeem to less than fees Eg: totalShares = 50 totalDelegated = 95 totalProfit = 10 fee = 50% fee obtained should be 50% of 10 == 5 but with the current calculation, fee obtained will be: fee shares = $5 * 50 / (95 + 10) == 2.38095238$ redeeming which will give $2.38095238 * 105 / (50 + 2.38095238) == 4.772$
Recommendations	Consider computing the feeShares without including the fees into totalDelegated
Comments / Resolution	Acknowledged.

Issue_06	Users might be unintentionally bricked from requesting a withdrawal
Severity	Medium
Description	When depositing there's no minimum constraint on the BNB amount or the <code>slisBnb</code> minted except that <code>slisBnb</code> minted shouldn't be zero . A normal user can therefore make a deposit of a small amount of BNB and when the user would request a withdrawal through <code>requestWithdraw()</code> the execution would revert due to <code>bnbToWithdraw</code> being less than <code>minBnb</code> → <pre> function requestWithdraw(uint256 _amountInSlisBnb) external override whenNotPaused { if (_amountInSlisBnb == 0) revert ErrorsLib.InvalidSlisBnbAmount(); uint256 bnbToWithdraw = convertSnBnbToBnb(_amountInSlisBnb); if (bnbToWithdraw <= minBnb) revert ErrorsLib.AmountTooSmall(); </pre> Therefore , the user is unintentionally and indefinitely blocked from requesting a withdrawal for his deposit.
Recommendations	Consider having a <code>minBnb</code> check while depositing too.
Comments / Resolution	Acknowledged.

Issue_07	delegateVoteTo will not work if there's no current delegator for govBNB
Severity	Medium
Description	Consider the following scenario → 1.) BOT delegates some BNB to a validator but currently <code>delegateVotePower</code> is switched off (is false). 2.) This means there's no current <code>govBNB</code> delegatee but there are <code>govBNB</code> tokens held by the <code>ListaStakeManager</code> contract. 3.) Now , when the admin invokes <code>delegateVoteTo()</code> the <code>currentDelegatee</code> would be zero address and the <code>currentDelegateeChange</code> would be 0 since → <pre> if (_delegateTo != address(this)) { require(govToken.getVotes(address(this)) == 0, "Invalid Delegation"); uint256 currDelegateeChange = currentVotePower - govToken.getVotes(currentDelegatee); currentVotePower is 0 and getVotes would also return 0. </pre> 4.) This results in <code>currentVotePower</code> not being equal to <code>balance</code> or <code>newDelegateeChange</code> and the <code>delegateVoteTo</code> reverts.
Recommendations	If there's no current delegatee assigned then the checks can be relaxed.
Comments / Resolution	Acknowledged. A self delegation will be performed initially as a work around.

Issue_08	BOT can be gas grieved when calling <code>claimWithdrawFor</code>
Severity	Low
Description	When a claim is ready to be claimed and users haven't claimed them yet , the BOT can call <code>claimWithdrawFor()</code> for such users to settle the pending claims. An attacker can leverage this functionality to have multiple claims pending and then when the BOT invokes the <code>claimWithdrawFor()</code> on attacker's behalf for all his claims the function would send native BNB to the attacker's address and that address can be a smart contract which consume all but some gas (or maybe even consumes all gas) in its <code>receive()</code> or <code>fallback()</code> function resulting in the BOT being gas grieved , moreover this can be done multiple times for multiple claims hence the BOT can be gas grieved continuously.
Recommendations	When transferring native BNB to the user specify a max gas amount that should be forwarded.
Comments / Resolution	Acknowledged.

Issue_09	Function redelegate() does not validate dstValidator whitelist
Severity	Low
Description	Function redelegate() performs a check to ensure the srcValidator is whitelisted. However, the same check is not performed for the dstValidator. This enables redelegations to occur from whitelisted validator A to non-whitelisted validator B. <pre>if (!validators[srcValidator]) revert ErrorsLib.InactiveValidator();</pre> A similar instance exists in the delegateVoteTo() function, where the _delegateTo parameter is not validated to be in the whitelist. <pre>function delegateVoteTo(address _delegateTo) external override onlyRole(DEFAULT_ADMIN_ROLE) {</pre>
Recommendations	Add a check to ensure dstValidator is in the whitelist.
Comments / Resolution	Fixed by implementing the recommendation.

Issue_10	Rounding down in getPooledBNBByShares() and getSharesByPooledBNB() bricks validator removals
Severity	Low
Description	Functions getPooledBNBByShares() and getSharesByPooledBNB() exist in validator credit contracts and perform regular solidity rounding down behaviour during exchange rate calculations. Function convertBnbToShares() in ListaStakeManager utilizes function getSharesByPooledBNB() particularly and convertBnbToShares() is further utilized by function redelegate(). Due to the rounding down behaviour, it is possible for the full amount to not be delegated to the new dstValidator. The remaining rounded down amount is left back with the srcValidator. When attempting to remove the source validator, it will fail since the validator still holds dust amount of shares and pooled BNB. For example, let's use the following example credit contract to demonstrate this. If we pass in 1e18 (1000000000000000000) to getPooledBNBByShares() as the share amount (redelegation amount), it would evaluate as 1017642985893128663 at the time of writing due to the current exchange rate. When this value is now passed to the getSharesByPooledBNB() functionality (which is used by the BNB stake hub and credit contracts internally as part of redelegation), it will provide the resultant shares as 999999999999999999. This means 1 wei of share token still exists with the source validator. If we pass in 1 wei to getPooledBNBByShares(), it returns 1 wei of BNB. Due to this, validator removals cannot occur as getDelegated() would return 1 wei of BNB to removeValidator() and the require check reverts. <pre>require(getDelegated[_address] == 0, "Balance is not zero");</pre>

```
function getDelegated(address _validator) public view override
returns (uint256) {
    address creditContract =
    IStakeHub[STAKE_HUB].getValidatorCreditContract(_validator);
    return
    IStakeCredit[creditContract].getPooledBNB(address(this))
    + IStakeCredit[creditContract].lockedBNBs(address(this), 0);
}
```

To note, this also affects undelegations as due to the rounding down, the undelegated amount becomes slightly lesser than the requested amount by the user, which could brick withdrawals. However based on the natspec comments for function `setReserveAmount()`, we assume this will be managed by the `DEFAULT_ADMIN_ROLE` and the operational overhead required for this is an acknowledged risk.

```
/**
 * @dev Adjust reserve amount. `reserveAmount` is the buffer for
 * undelegation.
 * Since the actual undelegate amount is slightly smaller than
 * the Bot requested amount due to precision loss,
 * Bot will add some extra BNB when calling undelegateFrom for
 * the first time, in order to cover the last request.
 * @param amount - Amount of Bnb
 */
function setReserveAmount(uint256 amount) external override
onlyRole(DEFAULT_ADMIN_ROLE) {
    reserveAmount = amount;
    emit SetReserveAmount(amount);
}
```

Recommendations

Consider acknowledging this by ensuring that this will be handled with the `reserveAmount` functionality and undelegating the dust amount. Alternatively, we recommend relaxing the `getDelegated()` require check during validator removals in such a case where the delegated amount is extremely small.

Comments / Resolution	Acknowledged.
-----------------------	---------------

Issue_11	Profits might not be uniformly distributed due to locked withdrawal amount on <code>requestWithdraw</code>
Severity	Low
Description	Considering a scenario where the BOT triggers an undelegation after every 5 withdrawal requests , users would only make a profit if <code>compoundRewards()</code> was called before they called <code>requestWithdraw</code> (because <code>compoundRewards</code> increases value of <code>slisBnb</code> shares instantly) , so assuming that out of 5 withdrawal requests 2 were made before compound reward and 3 were made after in this case the first 2 request would not experience any profit, this is because their request is queued with an “amount” which has been calculated according to the pre inflated rate i.e. pre profit , so the profit will only be experienced by the 3 users , while ideally when the BOT batches the undelegation and the claim is ready all 5 users should experience a slice of the profit. Moreover , a malicious validator can reorg the transaction order such that the <code>requestWithdraw()</code> tx is executed before <code>compoundRewards()</code> which leads to the user receiving 0 profits.
Recommendations	The design choice should be acknowledged.
Comments / Resolution	Acknowledged.

Issue_12	Possible storage collision with <code>maxBufferSizePct</code>
Severity	Low
Description	The old ListaStakeManager contract has a deprecated <code>placeholder</code> variable. <pre>// Deprecated variable uint256 public placeholder;</pre> This has now been replaced with the <code>maxBufferSizePct</code> variable. <pre>// The max buffer pool size percentage of 'totalPooledBnb' uint256 public maxBufferSizePct;</pre> This means that if the slot already held a non-zero uint256 value from the previous stake manager versions, it could corrupt the initial value of <code>maxBufferSizePct</code>. Although the value can be overridden using <code>setMaxBufferSizePct()</code>, if there are any calls to <code>delegateTo()</code> before <code>setMaxBufferSizePct()</code> is called, the delegate functionality would use an incorrect buffer size, which leads to more or less tokens being allocated to the buffer and delegated. This breaks an invariant of the protocol.
Recommendations	Set the value of <code>maxBufferSizePct</code> in the <code>initialize()</code> function itself
Comments / Resolution	Fixed by implementing the recommendation.

Issue_13	Outdated address stored in manager variable if MANAGER_ROLE is updated
Severity	Low
Description	Currently the contract is expected to assign the MANAGER_ROLE to the address stored in the manager state variable. However, if the MANAGER_ROLE is revoked from the original manager and assigned to another address through grantRole() by the DEFAULT_ADMIN_ROLE, the manager state variable still continues to store the previous manager's address
Recommendations	Implement functionality that allows the DEFAULT_ADMIN_ROLE to update the manager state variable in case a new manager is set.
Comments / Resolution	Acknowledged.

Issue_14	Maintain consistency on minimum withdrawal checks to ensure successful withdrawals
Severity	Low
Description	Function requestWithdraw() implements the following check: <pre><i>if (bnbToWithdraw <= minBnb) revert ErrorsLib.AmountTooSmall();</i></pre> While function instantWithdraw() implements the following check: <pre><i>if (bnbAmount < minBnb) revert ErrorsLib.AmountTooSmall();</i></pre> Since requestWithdraw() also considers the bnbToWithdraw == minBnb case, this could prevent user withdrawals that request exactly the minBnb amount of tokens, which is allowed in instantWithdraw().
Recommendations	In requestWithdraw() update the operator to < instead of <=
Comments / Resolution	Fixed by implementing the recommendation.

Issue_15	<code>_burnRefundSlisBnb</code> is performed after minting the fee shares causing the <code>revenuePool</code> to immediately obtain a portion of the refund
Severity	Low
Description	The revenue pool is supposed to earn a percentage of the accrued rewards. This is accounted for by minting the corresponding amount of shares to the revenue pool. But since the commission refund is processed after this by increasing the share value of users (by burning protocol owned shares) the revenue pool also gains a portion of this commission refund. Also note that since the commission refund is handled by minting and burning of shares as well, the corresponding shares earn a certain portion from the accrued rewards as well
Recommendations	Invoke <code>_burnRefundSlisBnb</code> before calculating the fee shares
Comments / Resolution	Acknowledged.

Issue_16	Increasing <code>instantWithdrawFeeRate</code> can cause users to pay more fees than they expected
Severity	Low
Description	If there are pending <code>instantWithdraw</code> invoking txs of users and <code>instantWithdrawFeeRate</code> is increased, these users will end up paying more in fees than they had expected
Recommendations	Communicate the fee increase / add an additional parameter to the <code>instantWithdraw</code> function to safeguard against fee increase
Comments / Resolution	Acknowledged.

Issue_17	Increasing synFee can cause users to pay more fees than expected
Severity	Low
Description	synFee is updated without invoking the compoundRewards function. Hence increasing the fee will cause the users to pay more fees for the already accrued rewards than they had expected
Recommendations	Create and invoke an inner version of the compoundRewards function before the fee is updated. We also recommend providing the DEFAULT_ADMIN_ROLE with the option to provide a bool parameter to setSynFee() and implementing an if block in it to either make a call to compoundRewards() or skip it. This ensures synFee can be updated even when compoundRewards() reverts.
Comments / Resolution	Acknowledged.

Issue_18	<code>getSlisBnbWithdrawLimit</code> implementation is incorrect
Severity	Low
Description	The amount available to withdraw is calculated as <code>totalDelegated - amountToUndelegate - unbondingBnb</code>. This is incorrect as it double counts the <code>undelegatedQuota</code> portion of the assets <pre> function getSlisBnbWithdrawLimit() external view override returns (uint256 _slisBnbWithdrawLimit) { uint256 amountToUndelegate = getAmountToUndelegate(); _slisBnbWithdrawLimit = convertBnbToSnBnb(totalDelegated - amountToUndelegate - unbondingBnb); } </pre>
Recommendations	Change to <pre> _slisBnbWithdrawLimit = convertBnbToSnBnb(totalDelegated - amountToUndelegate - unbondingBnb - undelegatedQuota) </pre>
Comments / Resolution	Acknowledged.

Issue_19	Commission burning doesn't follow daily schedule accurately
Severity	Low
Description	Commission burning is limited to 1 burn per invocation. Hence even if multiple days have passed since the last invocation, only the amount for a single day will be burned. Also whenever a new commission is added via <code>refundCommission</code> , the currently pending burn is not performed and the <code>block.timestamp</code> is not updated as well
Recommendations	Acknowledge / change the <code>_burnRefundSlisBnb</code> function to support burning of multiple days
Comments / Resolution	Acknowledged.

Issue_20	Unnecessary operation in <code>removeValidator</code>
Severity	Informational
Description	<code>removeValidators()</code> requires <code>validators[_address]</code> to be false which will be done via a prior call to <code>disableValidator()</code> , therefore the delete operation done at the end of <code>removeValidators()</code> i.e. <code>delete validators[_address];</code> is unnecessary.
Recommendations	The delete operation in <code>removeValidators()</code> can be removed.
Comments / Resolution	Acknowledged.

Issue_21	MANAGER_ROLE is not assigned to manager state variable
Severity	Informational
Description	The manager state variable is set in function initialize() however the same address is not assigned the MANAGER role on initialization. <pre>manager = _manager;</pre> While the default admin role can separately assign this role to the address, it is recommended to assign it during initialization to maintain consistency.
Recommendations	Assign the MANAGER_ROLE to the manager address during initialization itself.
Comments / Resolution	Fixed by implementing the recommendation.

Issue_22	Function instantWithdraw() can be optimized to save gas
Severity	Informational
Description	Currently in function instantWithdraw(), SLisBnb tokens are first transferred in from the user to the contract and followingly the tokens are burned from it. <pre>IERC20Upgradeable[slisBnb].transferFrom(msg.sender, address(this), _amountInSlisBnb);</pre> <pre>ISLisBNB[slisBnb].burn(address(this), burnAmount);</pre> However, this two step process can be optimized to a single statement where the slisBnb tokens are directly burned from the msg.sender.
Recommendations	As mentioned, consider burning the tokens directly from msg.sender

Comments / Resolution	Acknowledged. The design is intended as two-step so we charge the fee in slisBnb.
------------------------------	---

Issue_23	Unnecessary check in delegateVoteTo()
Severity	Informational
Description	Function delegateVoteTo() does not require the following require check as the delegate call cannot fail or incorrectly assign another address other than _delegateTo <pre>govToken.delegate(_delegateTo); require(govToken.delegates(address(this)) == _delegateTo, "Delegation Failed");</pre>
Recommendations	Remove the require check
Comments / Resolution	Fixed by implementing the recommendation.

Issue_24	Theoretical share price inflation/incorrect valuation
Severity	Informational
Description	share inflation (eg: can occur if all shares are withdrawn but newer rewards remain making totalAssets non-zero) and inflated/deflated valuation (eg: lowered underlying value due to reallocation fees) are all possible in the current implementation under extreme scenarios. And if this happens it is possible for the users to lose their assets (eg: if share value is inflated with totalShares == 1, then significant value can be lost if the deposit is not made in multiples of existing totalAssets)
Recommendations	Can be guarded against by ensuring that the protocol maintains a deposit/shares themselves and performs reallocations ensuring that it doesn't decrease the overall value enough to lower the share price
Comments / Resolution	Acknowledged.